Object Detection & Localization: Architecture, Models and Edge vs. Cloud Deployment (2026)

Oct 26, 2026 · @Art

Executive summary

For finding objects with precise coordinates in 2026, use a **single-stage real-time detector** — YOLO26 (CNN) or RF-DETR / DEIMv2 (DETR-style with a ViT backbone) — not a plain classification CNN such as AlexNet, VGG or ResNet. Both families run on Qualcomm NPUs and other edge accelerators after INT8 quantization; cloud GPUs are only needed for large, open-vocabulary or foundation models.

- **Approximate location** ("is it there, roughly where?"): a classifier with a heatmap (CAM), a grid/centroid detector such as FOMO, or a tiny detector at low resolution. Fits even microcontrollers.
- **Precise coordinates** (bounding boxes `x1, y1, x2, y2` or `cx, cy, w, h`): YOLO26-n/s or RF-DETR-N/S on edge; YOLO26-l/x, RF-DETR-L/2XL or DEIMv2-X on a GPU server.
- **Pixel-precise outlines**: instance segmentation (YOLO26-seg, RF-DETR-Seg); SAM 3 for zero-shot "find every X" in the cloud.
- **Qualcomm edge**: a nano-size YOLO runs in about 1–5 ms on the Hexagon NPU of mid/high-tier Snapdragon/Dragonwing chips (INT8), and about 5–16 ms on the entry-level QCS6490.
- **Cloud only when** you need open-vocabulary detection (text prompts), very high accuracy on tiny objects at high resolution, or central multi-camera analytics.
- **Licence matters**: Ultralytics YOLO is AGPL-3.0 (commercial licence needed for closed products); RF-DETR (N–L), DEIM/DEIMv2 and D-FINE are Apache-2.0.

1. Which task do you actually need?

The precision of "where" decides the model family more than anything else. Pick the lowest row that answers your business question — each step down costs more labelling, compute and latency.

Task	Output	Localization precision	Typical models	Labelling cost	Smallest hardware
Image classification	One label per image	None	ResNet, MobileNetV3/V4, EfficientNet, YOLO26-cls	Very low (image tag)	MCU (Cortex-M7)
Classification + heatmap (CAM / Grad-CAM)	Label + coarse heat region	Approximate (blob)	Any CNN classifier + CAM	Very low	MCU / low-end NPU
Grid / centroid detection	Class + object centre per grid cell	Approximate (point, ~1/8 of input resolution)	Edge Impulse FOMO, CenterNet-style heads	Low (boxes or points)	MCU, under 200 KB RAM
Object detection (boxes)	Class + box `x1,y1,x2,y2` + confidence	Precise (pixel coordinates of a rectangle)	YOLO26, RF-DETR, DEIMv2, D-FINE, RT-DETR, Faster R-CNN	Medium (boxes)	Low/mid NPU (QCS6490, Hailo-8L)
Oriented detection (OBB)	Rotated box + angle	Precise, rotation-aware	YOLO26-obb	Medium	Mid NPU
Instance segmentation	Box + pixel mask per object	Pixel-precise outline	YOLO26-seg, RF-DETR-Seg, Mask R-CNN	High (polygons)	Mid/high NPU, Jetson
Keypoints / pose	Box + named points	Precise points on the object	YOLO26-pose, RF-DETR Keypoint	High	Mid NPU
Open-vocabulary detection / segmentation	Boxes/masks for any text prompt	Precise, classes not fixed	SAM 3, YOLOE-26, YOLO-World, Grounding DINO, OWLv2	None to low (zero-shot)	Cloud GPU (SAM 3); high-end NPU (YOLOE/YOLO-World)

Rule of thumb: "Is a pallet in the frame and roughly where?" → centroid detection. "Where exactly, so a robot can pick it or a measurement can be taken?" → box detection. "What is the exact shape / area?" → segmentation.

2. Backbones: AlexNet, VGG, ResNet and what replaced them

AlexNet, VGG and ResNet are **classification backbones** — feature extractors, not detectors. They tell you *what* is in the image; a detection head and neck (FPN/PAN, anchors or queries) are needed on top to get *where*. Today ResNet survives as a reference and in Faster/Mask R-CNN; AlexNet and VGG are obsolete for new products. Parameter and FLOP figures below are the commonly cited ImageNet values at 224×224 (approximate).

Backbone	Year	Params (M, approx.)	GFLOPs @224 (approx.)	Role today	Edge suitability
AlexNet	2012	61	0.7	Historical; teaching only	Poor (large FC layers, weak accuracy)
VGG-16	2014	138	15.5	Legacy SSD backbone; style transfer	Poor (huge, slow)
ResNet-50	2015	25.6	4.1	Faster/Mask R-CNN, DETR baselines, MLPerf reference	Fair on NPUs after INT8; heavy for MCUs
MobileNetV2/V3/V4	2018–2024	2–6	0.06–0.6	SSDLite, mobile classifiers	Excellent; NPU-friendly depthwise convs
EfficientNet / EfficientDet	2019–2020	4–66	0.4–37	Compound scaling; superseded by YOLO/DETR	Good (B0–B2)
CSPDarknet / C2f / C3k2 (YOLO)	2020–2026	2–56 (whole detector)	5–195 (detector @640)	YOLOv5 → YOLO26	Excellent; built for export
HGNetv2	2023	1–60	—	RT-DETR, D-FINE, small DEIMv2 variants	Very good
ConvNeXt	2022	28–198	4.5–34	Modern pure CNN, server detectors	Fair
ViT / DINOv2 / DINOv3	2020–2025	22–300+	varies	RF-DETR (DINOv2), DEIMv2 (DINOv3), SAM 3	Small variants OK on GPU/Jetson; harder on NPUs (attention ops)

Takeaway: don't build "ResNet + custom head" yourself. Take a complete, pretrained detector family (YOLO26, RF-DETR, DEIMv2) and fine-tune it — you get the backbone, neck, head, loss, augmentations and export tooling together.

3. Detection algorithm families

The field has converged on two winners for production: **one-stage CNN detectors (YOLO)** and **real-time DETRs** (transformer decoders with object queries). Both are now NMS-free end to end, which simplifies deployment. Two-stage detectors remain for niche high-precision work; open-vocabulary models are for flexibility, not speed.

Family	How it localizes	Representative models	Strengths	Weaknesses	Use in 2026
Two-stage	Region proposals (RPN), then classify + refine each box	Faster R-CNN, Mask R-CNN, Cascade R-CNN	Precise boxes, strong on small objects, mature	Slow (10–30 FPS on GPU), complex export	Offline analysis, research baselines
One-stage, anchor-based	Dense grid of predefined anchor boxes + regression	SSD, RetinaNet, YOLOv3–v5	Fast, simple	Anchor tuning, NMS needed	Legacy; MCU-class SSDLite
One-stage, anchor-free CNN	Each grid cell predicts distances to box edges / centre	FCOS, CenterNet, YOLOv8, YOLO11, **YOLO26**	Best speed/accuracy on CPU and NPU, best tooling	AGPL licence (Ultralytics)	**Default for edge**
Real-time DETR (transformer)	Learned object queries attend to image; set prediction, no NMS	RT-DETR, D-FINE, DEIM/DEIMv2, LW-DETR, **RF-DETR**	Highest accuracy per ms on GPU, strong domain transfer, Apache-2.0	Attention ops less mature on some NPUs; heavier training	**Default for GPU / Jetson; strong on edge**
Grid/centroid (MCU)	Per-cell class heatmap → object centroids	Edge Impulse FOMO	Runs in <200 KB RAM	No box size; objects must be similar size and not overlap	Microcontrollers
Open-vocabulary	Text (or image) prompt embedded and matched to regions	YOLO-World, YOLOE-26, Grounding DINO, OWLv2, **SAM 3**	No fixed class list, zero-shot	Lower accuracy than fine-tuned specialists; SAM 3 is 848M params	Auto-labelling, prototyping, cloud analytics

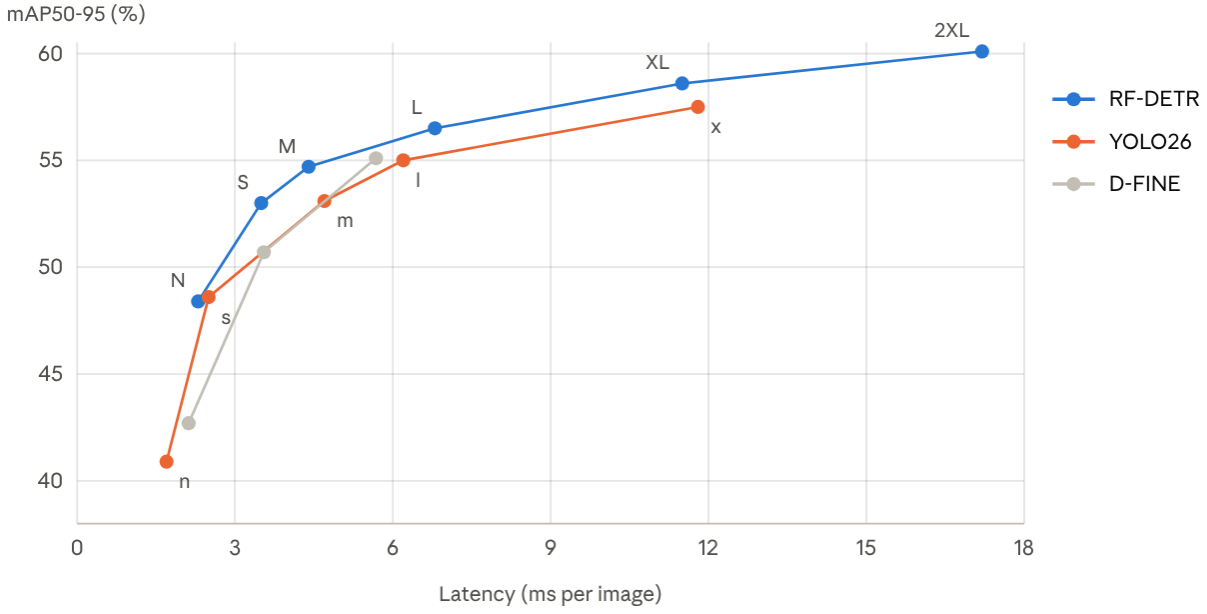
Non-maximum suppression (NMS) used to be a deployment pain point (variable latency, NPU-unfriendly). YOLOv10, YOLO26's one-to-one head and every DETR remove it, so the latency you benchmark is the latency you get.

4. State-of-the-art models (October 2026)

The accuracy frontier is now held by DETR-style models with self-supervised ViT backbones (RF-DETR, DEIMv2); YOLO26 is the best pure-CNN option and the most edge-friendly. RF-DETR-2XL is the first real-time detector above 60 COCO mAP.

RF-DETR leads accuracy per millisecond; YOLO26 is fastest at the small end

COCO val mAP50-95 vs. latency per image, NVIDIA T4, TensorRT FP16, batch 1



Ultralytics YOLO26 docs; RF-DETR docs and Roboflow edge benchmark (RF-DETR, D-FINE) · as published 2026

Latency protocols differ slightly between vendors (Ultralytics measures YOLO26's NMS-free head; Roboflow measures RF-DETR and D-FINE end to end), so treat gaps under ~0.5 ms as noise. COCO rank is not your dataset — RF-DETR's DINOv2 backbone transfers better to unusual domains (RF100-VL), so benchmark on your own images.

Model family	Sizes	Params (M)	Input (px)	Licence	Tasks	Best fit
YOLO26 (Ultralytics, Jan 2026)	n / s / m / l / x	2.4 – 55.7	640	AGPL-3.0 or paid Enterprise	Detect, seg, pose, OBB, cls, depth; YOLOE-26 open-vocab	Edge NPUs, CPUs, MCU-class SoCs; fastest tooling
RF-DETR (Roboflow, ICLR 2026)	N / S / M / L / XL / 2XL	30.5 – 126.9	384 – 880	Apache-2.0 (N–L)	Detect, seg, keypoints (preview)	Jetson / GPU, custom domains, commercial closed-source
DEIMv2 (DINOv3)	Atto … X (8 sizes)	0.5 – 50.3	640	Apache-2.0	Detect	Ultra-light (Pico 1.5M = 38.5 AP) up to X (57.8 AP)

Model family	Sizes	Params (M)	Input (px)	Licence	Tasks	Best fit
D-FINE / DEIM	N / S / M / L / X	4 – 62	640	Apache-2.0	Detect	Strong DETR baseline; DEIM-D-FINE-X 56.5 AP at 78 FPS on T4
YOLO11 / YOLOv8	n … x	2.6 – 57	640	AGPL-3.0	Detect, seg, pose, OBB	Widest export support on older SDKs (e.g. Qualcomm AI Hub, Hailo model zoo)
SAM 3 (Meta, Nov 2025)	one	848	1008	SAM licence (gated)	Open-vocab detect + seg + video tracking	Cloud: auto-labelling, "find every X" by text
Faster / Mask R-CNN (ResNet-50/101 FPN)	—	41 – 63	~800–1333	Apache-2.0 (torchvision/Detectron2)	Detect, seg	Offline high-precision, research
FOMO (Edge Impulse)	—	<0.1	96 – 160	Edge Impulse terms	Centroids only	Microcontrollers

5. Running on IoT / edge devices

Yes — precise box detection runs comfortably on edge hardware, including Qualcomm. A nano/small YOLO quantized to INT8 runs in single-digit milliseconds on any NPU-equipped Snapdragon or Dragonwing chip. What does *not* fit on typical IoT hardware: SAM 3 (848M params), Grounding DINO-class open-vocabulary models, and large DETRs at high resolution.

5.1 Measured latency by device tier

Device tier	Example hardware	Accelerator	Measured detector latency	Practical model ceiling	Power / price
Microcontroller	Arduino Nicla Vision (Cortex-M7)	None (CPU, INT8)	FOMO ~30 FPS using 245 KB RAM	Centroid detection only, 96–160 px	<1 W, $
Entry IoT SoC (Qualcomm)	Dragonwing QCS6490	Hexagon NPU	YOLO11-N 640px: **4.9 ms** (INT8 w8a8, TFLite); 16.3 ms (w8a16, ONNX)	YOLO26 n/s; open-vocab YOLO-World ~193 ms (too slow for video)	Low W

Device tier	Example hardware	Accelerator	Measured detector latency	Practical model ceiling	Power / price
Mid/high IoT SoC (Qualcomm)	QCS8550, IQ-9075, IQ-8275	Hexagon NPU	YOLO11-N: **1.6–1.9 ms** (w8a8)	YOLO26 s/m; RF-DETR N/S if ops supported	Low–mid W
Flagship mobile (Qualcomm)	Snapdragon 8 Gen 3 / 8 Elite / 8 Elite Gen 5	Hexagon NPU	YOLO11-N: **0.7–1.1 ms** (w8a8)	YOLO26 m/l; YOLO-World ~61 ms	Phone-class
Raspberry Pi + NPU HAT	Pi 5 + AI HAT+ (Hailo-8, 26 TOPS, $110)	Hailo-8	YOLOv8s ~80 FPS; YOLOv8m 50–60 FPS (Pi 5 CPU alone: YOLOv8n 5–8 FPS)	YOLO s/m; AI HAT+ 2 (Hailo-10H, 40 TOPS, $130) runs YOLOv8l at 30–40 FPS	+~4.5 W
Embedded GPU	Jetson Orin Nano Super 8 GB (67 TOPS)	Ampere GPU, TensorRT	YOLOv8n: **3.5 ms** INT8 / 4.4 ms FP16	YOLO26 m/l, RF-DETR N–M, segmentation	7–25 W, ~$250
Embedded GPU (upper)	Jetson Orin NX 16 GB	Ampere GPU, TensorRT	YOLO26s: **5.95 ms** INT8 / 7.94 ms FP16	RF-DETR L, multi-camera	10–25 W
x86 CPU only	Intel Xeon 2.0 GHz	None (ONNX Runtime)	YOLO26n: 38.9 ms; YOLO26s: 87.2 ms	n/s at a few FPS per core	—
Cloud GPU	NVIDIA T4 / L4 / A100 / H100	TensorRT	YOLO26x 11.8 ms; RF-DETR-2XL 17.2 ms (T4)	Anything incl. SAM 3, Grounding DINO	Pay per hour

Qualcomm figures are Qualcomm AI Hub profiles of YOLO11-N (2.64M params, 640×640; INT8 model is 2.83 MB). YOLO26-n is similar size (2.4M params) and expected to land in the same range; profile it yourself on AI Hub before committing.

5.2 Qualcomm-specific workflow

1. **Train/fine-tune** in PyTorch (Ultralytics or RF-DETR) on a cloud GPU.
2. **Export** to ONNX (for YOLO26 use `nms=False` to get the NMS-free head — friendlier to the NPU).
3. **Compile and quantize** with Qualcomm AI Hub (`qai-hub-models`) to TFLite, QNN context binary/DLC, or ONNX Runtime QNN EP; use w8a8 for speed, w8a16 if INT8 hurts accuracy.
4. **Profile on hosted real devices** (QCS6490, QCS8550, IQ-9075, Snapdragon phones) in AI Hub — no hardware needed to choose a chip.

5. **Deploy** with the Qualcomm Intelligent Multimedia SDK / GStreamer (IoT) or LiteRT/QNN in an Android app; keep pre/post-processing (resize, letterbox, decode) on the DSP/GPU where possible.

5.3 What makes a model edge-friendly

- **INT8 quantization**: typically 2–4× faster and 4× smaller than FP32 for roughly 0.5–2 mAP loss with post-training quantization; use quantization-aware training if the loss is larger.
- **Static shapes and NMS-free heads** (YOLO26 `nms=False`, DETRs): fully on the NPU, predictable latency.
- **Plain convolutions over attention**: CNN YOLOs have the broadest NPU operator coverage; transformer DETRs run best on GPUs (Jetson, cloud) and need operator-support checks on Hexagon/Hailo.
- **Resolution is the biggest lever**: halving input from 640 to 320 cuts compute about 4×, at the cost of small objects. Tile the image or add a P2 head (YOLO26-p2) when objects are tiny.

6. Edge vs. cloud vs. hybrid

For most detection products the answer is **edge inference with cloud training** — or a hybrid where the device filters and the cloud handles hard cases. Pure cloud inference only wins when the model is too large for the device or classes change constantly.

Criterion	On-device (edge / IoT)	Cloud inference	Hybrid (edge filter + cloud verify)
Latency	1–30 ms, no network	50–500 ms incl. upload, network-dependent	Edge-fast for common cases; cloud only on triggers
Bandwidth	Only metadata (boxes, counts) leaves the device	Full images/video uploaded	Crops or keyframes only
Privacy / GDPR	Images never leave the device	Personal data in transit and at rest	Minimised; blur/crop before upload
Offline operation	Yes	No	Degrades gracefully

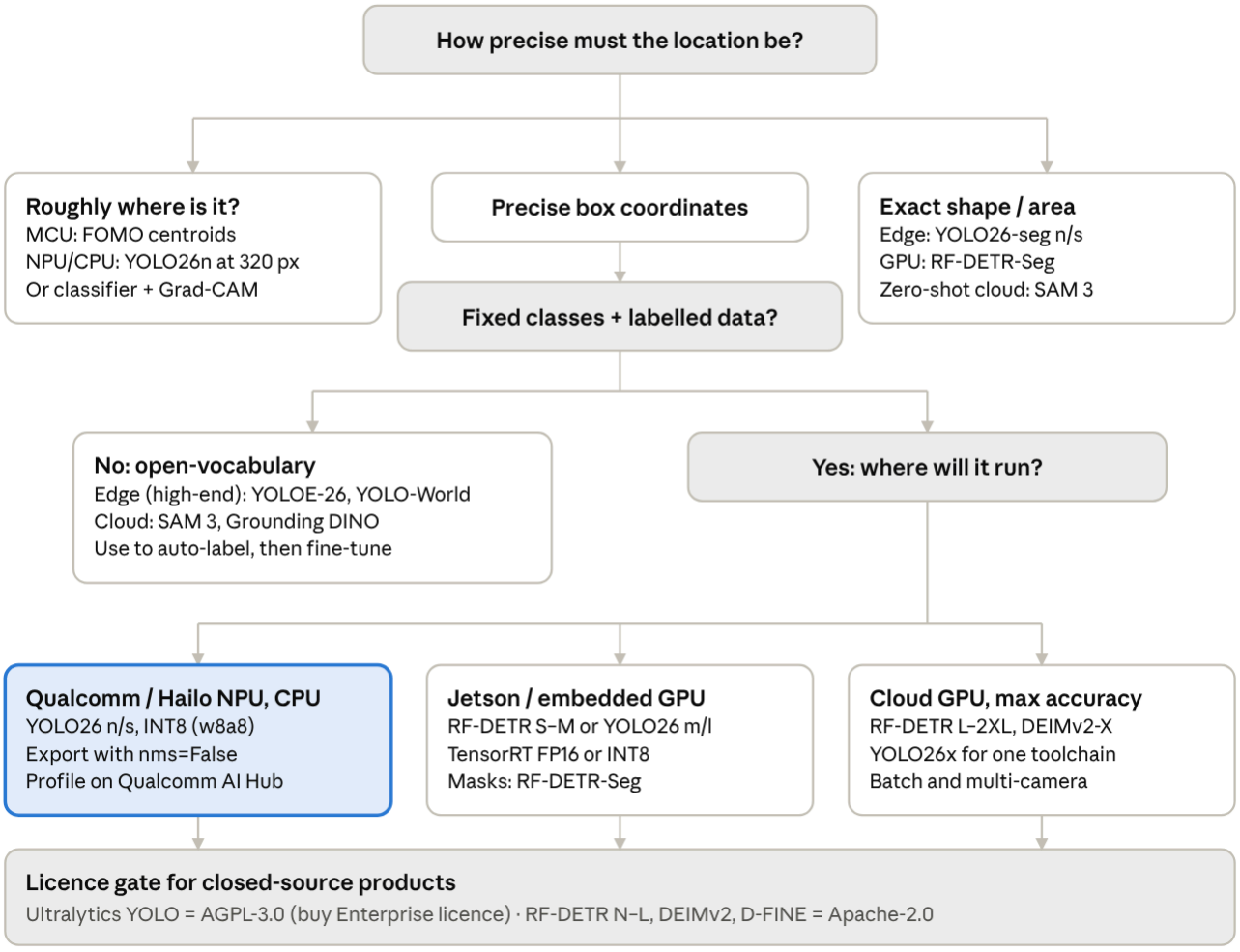
Criterion	On-device (edge / IoT)	Cloud inference	Hybrid (edge filter + cloud verify)
Model size ceiling	~1–60M params (INT8)	Unlimited (SAM 3, VLMs)	Small on edge, large in cloud
Open-vocabulary / changing classes	Limited (YOLOE/YOLO-World on high-end chips)	Yes, zero-shot	Yes, via cloud fallback
Cost profile	Hardware up front (CAPEX), ~zero per inference	Per-hour GPU (OPEX), scales with traffic	Mixed; cloud cost a fraction of pure cloud
Updates	OTA model push, per-device fleet management	Instant, one place	Both
Best for	Cameras, robots, drones, retail shelves, industrial QA, smartphones	Batch analytics, auto-labelling, rare-class search, low-volume APIs	Security alerts, quality inspection with human review, active learning

Training always needs GPU compute (cloud or a local workstation). A YOLO26-s or RF-DETR-S fine-tune on a few thousand images takes hours on a single modern GPU; edge devices only run inference.

7. Decision roadmap

Answer three questions in order — precision, class flexibility, hardware — then check the licence. The highlighted box is the default for a Qualcomm-class IoT product.

Choose by location precision, then class flexibility, then hardware



decision roadmap · 3 questions, 7 outcomes, 1 licence gate

If two branches both fit, prototype both: fine-tune YOLO26-s and RF-DETR-S on the same 500–1,000 labelled images, export both to the target device, and pick by measured mAP on your validation set and measured on-device latency.

Your requirement	Threshold	Choose
Latency budget per frame	< 5 ms on NPU	YOLO26-n (INT8)
Latency budget per frame	5–30 ms	YOLO26-s/m or RF-DETR-N/S
Accuracy target (COCO-style mAP)	> 58	Cloud/GPU: RF-DETR-XL/2XL
Power	< 1 W	FOMO on MCU, or wake-on-motion + NPU
Small objects (< 16 px)	any	Higher resolution, tiling, YOLO26-p2 head

Your requirement	Threshold	Choose
Data available	< 100 images per class	Auto-label with SAM 3 / YOLOE, then fine-tune
New classes added often	weekly or more	Open-vocabulary (YOLOE-26 edge, SAM 3 cloud)
Closed-source commercial product	—	Apache-2.0 models or Ultralytics Enterprise licence

8. Implementation plan

A typical path from idea to fleet takes 8–12 weeks; the gate at each step is a measured number, not a guess.

1. **Define the requirement (week 1)** — localization precision, class list, latency budget, target device, licence constraints. Collect 200–500 representative images from the real camera, including hard cases (night, blur, occlusion).
2. **Bootstrap labels (weeks 1–2)** — auto-label with SAM 3 or YOLOE-26 text prompts, then correct by hand. Split train/val/test by location or time, not randomly.
3. **Baseline both families (weeks 2–3)** — fine-tune YOLO26-s and RF-DETR-S from COCO weights on a cloud GPU. Gate: validation mAP50-95 and per-class recall meet the business target.
4. **Port to the device (weeks 3–5)** — export ONNX (`nms=False` for YOLO26), quantize INT8 (Qualcomm AI Hub / TensorRT / Hailo Dataflow Compiler), profile on the real chip. Gate: on-device latency within budget and INT8 mAP loss under ~2 points.
5. **Pilot (weeks 5–8)** — run on 5–20 devices; log low-confidence frames to the cloud (hybrid pattern) for review and relabelling.
6. **Production and MLOps (week 8+)** — OTA model updates, drift monitoring, periodic retraining with the hard cases collected in the pilot.

Sources

- Ultralytics YOLO26 documentation — architecture, COCO benchmarks, AGPL/Enterprise licence
- YOLO26 on NVIDIA Jetson (DeepStream/TensorRT) — Orin NX latency
- RF-DETR documentation and RF-DETR full docs — sizes, COCO/RF100-VL results, licence
- Roboflow: RF-DETR for the edge — RF-DETR vs D-FINE vs YOLO11 on T4

- DEIMv2: Real-Time Object Detection Meets DINOv3 and DEIM (CVPR 2025)
- Qualcomm AI Hub — YOLOv11-Detection profiles and YOLO-World on Qualcomm
- Jetson Orin Nano benchmarks — YOLOv8n TensorRT INT8/FP16
- Raspberry Pi AI HAT+ documentation and raspberry.tips benchmarks
- Edge Impulse FOMO announcement and FOMO tutorial
- SAM 3 (Ultralytics docs) and SAM 3 README

Backbone parameter/FLOP figures for AlexNet, VGG, ResNet, MobileNet, EfficientNet and ConvNeXt are commonly cited published values (approximate), not re-measured.